

Amortized Analysis

均摊复杂度

前置知识: [时间复杂度](#)

本页面将介绍均摊复杂度的基础知识。

引入

均摊分析 (Amortized Analysis) 是一种用于分析算法和动态数据结构性能的技术。它不仅仅关注单次操作的成本, 还通过评估一系列操作的平均成本, 为整体性能提供更加准确的评估。均摊分析不涉及概率, 且只能确保最坏情况性能的每次操作耗费的平均时间, 并不能确认系统的平均性能。在最坏情况下, 均摊分析通过将高成本操作的开销分摊到低成本操作上, 确保整体操作的平均成本保持在合理范围内。

均摊分析通常采用三种主要分析方法: 聚合分析、记账分析和势能分析。这些方法各有侧重, 分别适用于不同的场景, 但它们的共同目标是通过均衡操作成本, 优化数据结构在最坏情况下的整体性能表现。

内容

考虑一个可扩展的数组, 例如 C++ 中的 `vector`, 其初始容量为 n 。每次插入新元素时, 如果数组已满, 则需要将数组的大小加倍, 然后将原数组中的元素复制到新数组中, 最后插入新元素。

接下来, 将以动态数组的插入操作为例, 通过聚合分析、记账分析和势能分析三种方法, 分析其均摊成本。

聚合分析

聚合分析 (Aggregate Analysis) 通过计算一系列操作的总成本, 并将其平均到每次操作上, 从而得出每次操作的均摊时间复杂度。

以动态数组为例, 首先, 可以得到插入操作的两个关键成本:

- 如果数组未满, 插入操作的成本为 $O(1)$ 。
- 如果数组已满, 则插入操作需要扩容, 扩容后复制元素的成本为 $O(n)$, 其中 n 为当前数组的大小。

所以, 为了计算 n 次插入操作的总成本, 可以将其分开为两部分计算:

- 插入操作的成本:** 每次插入新元素的直接成本是常数时间, 对于 n 次操作, 总成本是 $O(n)$ 。
- 数组扩容的成本:** 每次扩容涉及到复制原数组元素到新数组。这些操作发生在数组大小为 n 的时刻, 其中 n 是小于等于 $2n$ 的最大幂。扩容操作的成本分别是 $O(n), O(2n), O(4n), \dots$, 总和为 $O(n)$, 这是一个等比数列的和, 其结果为 $O(n)$ 。

因此, 该数组总的插入成本为 $O(n)$, 均摊到每次操作的成本为 $O(1)$ 。即使在最坏情况下, 平均每次插入操作的成本依然是常数时间。

记账分析

记账法 (Accounting Method) 通过为每次操作预先分配一个固定的均摊成本来确保所有操作的总成本不超过这些预分配的成本总和。记账法类似于一种 **费用前置支付** 的机制, 其中较低成本的操作会存储部分费用, 以支付未来高成本的操作。

以动态数组为例, 可以为每次插入操作分配一个固定的均摊成本, 以确保在需要扩容时已经预留了足够的费用。

- 费用分配:**

- 假设每次插入操作的实际成本为 c ，均摊成本设为 $\bar{c}$ 。
- 其中 $\bar{c}$ 用于当前插入操作， c 用于未来可能的扩容操作。

2. 费用使用：

- 当数组已满时，需要进行扩容操作，实际成本为 c ，其中 n 是当前数组的大小。
- 假设扩容前数组的元素数量为 n ，由于原数组的后半部分 $\frac{n}{2}$ 个元素在插入时共预存了 $\frac{n}{2}$ 单位的均摊成本，恰好足够支付扩容操作的成本。

以下是一个具体的示例：

```

1 初始状态：
2 arr      = [1, 2, 3, 4] // 初始数组
3 amount   = [2, 2, 2, 2] // 每个元素预存的费用
4
5 // 第一轮扩容：数组已满，需要扩容
6 arr      = [1, 2, 3, 4, null, null, null, null] // 扩容后数组
7 amount   = [2, 2, 0, 0, 0, 0, 0, 0] // 3, 4的费用用于支付扩容
8
9 // 继续插入新元素，直至再次满载
10 arr     = [1, 2, 3, 4, 5, 6, 7, 8] // 继续填充数组
11 amount  = [2, 2, 0, 0, 2, 2, 2, 2] // 新插入的元素同样预存费用
12
13 // 第二轮扩容：数组再次满载，需要更大的空间
14 arr     = [1, 2, 3, 4, 5, 6, 7, 8, null, null, null, null, null, null, null, null] // 扩容后数组
15 amount  = [2, 2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0] // 5, 6, 7, 8的费用用于支付扩容

```

以上过程表明，每次插入操作所存储的均摊成本足够支付未来的扩容操作，从而确保了每次操作的均摊成本维持在 $\bar{c}$ 。

势能分析

势能分析（Potential Method）通过定义一个势能函数（通常表示为 Φ ），度量数据结构的 **潜在能量**，即系统状态中的预留资源，这些资源可以用来支付未来的高成本操作。势能的变化用于平衡操作序列的总成本，从而确保整个算法的均摊成本在合理范围内。

原理

首先，定义 **状态** 为某一时刻数据结构的状态，该状态可能包含元素数量、容量、指针等信息，其中定义初始状态为 S_0 ，即未进行任何操作时的状态。

其次，定义势能函数 Φ 用于度量数据结构状态 S 的势能，其满足以下两个性质：

1. **初始势能**：在数据结构的初始状态 S_0 下，势能 $\Phi(S_0) = 0$ 。
2. **非负性**：在任意状态 S 下，势能 $\Phi(S) \geq 0$ 。

对于每个操作，其均摊成本 $\bar{c}$ 定义为：

其中 c 为操作的实际成本， S_i 和 S_{i+1} 分别表示操作前后的数据结构状态。该公式表明，均摊成本等于实际成本加上势能的变化。如果操作增加了势能（即 $\Phi(S_{i+1}) > \Phi(S_i)$ ），则均摊成本上升；如果操作消耗了势能（即 $\Phi(S_{i+1}) < \Phi(S_i)$ ），则均摊成本下降。

我们可以通过势能函数来分析一系列操作的总成本。设 S_0 为从初始状态 S_0 开始，经过 n 次操作后产生的状态序列， c_i 为第 i 次操作的实际开销，那么第 i 次操作的均摊成本 $\bar{c}_i$ 为：

因此， n 次操作的总时间花销为：

由于，总时间花销的上界为：

因此，若，则 是均摊复杂度的一个上界。

示例：动态数组的扩容分析

以动态数组 `vector` 的插入操作为例，定义如下的势能函数：

其中， n 是数组中的元素数量， m 是数组的当前容量。这个势能函数反映了数组中剩余可用空间的数量，即当前容量和实际使用空间之间的差异。

1. 插入操作（无需扩容）：

- 操作成本： 1 ，因为只需插入一个元素。
- 势能变化：插入后，元素数量增加 1 ，势能增加 1 。
- 均摊成本： 1

2. 插入操作（触发扩容）：

- 假设当前容量为 m ，插入一个新元素时触发扩容，新的容量变为 $2m$ 。
- 操作成本： $2m$ ，因为需要将所有元素复制到新数组中，并插入新元素。
- 势能变化：扩容后，容量增加，势能减少，变化大小为 m 。
- 均摊成本： 1

通过上述分析可以看出，尽管扩容操作的实际成本较高，但由于势能函数的设计，整体均摊成本仍然保持在常数级别。

扩展示例：堆栈操作

堆栈操作是均摊分析的经典应用场景之一。假设堆栈 `s` 支持以下三种操作：

操作	说明	实际成本
<code>s.push(x)</code>	将元素 <code>x</code> 入栈	1
<code>s.pop()</code>	弹出栈顶元素	1
<code>s.multi-pop(k)</code>	弹出栈顶 <code>k</code> 个元素	k

我们将通过聚合分析、记账分析和势能分析三种方法来分析这些堆栈操作的均摊成本。

聚合分析法

聚合分析将计算所有操作的总成本，并将其平均分摊到每个操作上，从而得出均摊成本。

- 对于 n 次 `push(x)` 操作，每次的成本为 1 ，因此总成本为 n 。
- 对于 n 次 `pop()` 操作，每次的成本为 1 ，总成本为 n 。
- 对于 n 次 `multi-pop(k)` 操作，尽管每次的实际成本为 k ，但这些操作弹出的元素数量不会超过之前 `push(x)` 的元素数量，因此总成本仍受 n 的约束。

由于总操作次数为 $2n$ ，所以总成本为 $2n$ ，每次操作的均摊成本为 1 。

记账分析法

记账分析为每次 $\text{push}(x)$ 操作预留一部分费用，以支付未来可能的 $\text{pop}()$ 或 $\text{multi-pop}(k)$ 操作。

1. $\text{s.push}(x)$ ：假设每次 $\text{push}(x)$ 操作的均摊成本为 c ，其中 α 单位用于当前操作，另 $c - \alpha$ 单位存储为费用，用于支付未来的 $\text{pop}()$ 或 $\text{multi-pop}(k)$ 操作。
2. $\text{s.pop}()$ ：实际成本为 α ，但由于之前的 $\text{push}(x)$ 操作已为其预存了 $c - \alpha$ 单位费用，因此均摊成本为 c 。
3. $\text{s.multi-pop}(k)$ ：每个弹出的元素的实际成本为 α ，可以由之前该元素的 $\text{push}(x)$ 操作预存的费用支付，因此均摊成本为 c 。

通过以上分析，入栈操作预存的费用足以支付未来该元素的出栈操作，因此每次操作的均摊成本为 c 。

势能分析法

势能分析定义了一个势能函数来衡量堆栈的状态，并利用势能的变化来平衡操作成本。

1. **势能函数**：设 n 为堆栈中的元素数量，即 $|S|$ 。每个元素贡献 c 单位的势能。
2. $\text{s.push}(x)$ ：每次 $\text{push}(x)$ 操作增加堆栈中的元素数量，势能增加 c ，因此均摊成本为 c 。
3. $\text{s.pop}()$ ：每次 $\text{pop}()$ 操作减少堆栈中的元素数量，势能减少 c ，因此均摊成本为 c 。
4. $\text{s.multi-pop}(k)$ ： $\text{multi-pop}(k)$ 操作弹出 k 个元素，势能减少 kc ，因此均摊成本为 c 。

通过以上势能函数设计， $\text{push}(x)$ 操作的均摊成本为 c ，而 $\text{pop}()$ 和 $\text{multi-pop}(k)$ 操作的均摊成本为 c 。因此，所有堆栈操作的均摊成本均为 c 。

参考资料

- [Amortized Analysis - Wikipedia](#)
- [Cornell CS 3110 - Lecture 20: Amortized Analysis](#)

本页面最近更新：2024/9/23 09:40:36, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [HeRaNO](#), [yyyu-star](#)

本页面的全部内容 [在 CC BY-SA 4.0 和 SATA 协议之条款下](#) 提供，附加条款亦可能应用